Lista 02 de Exercícios — Estruturas de Repetição em C

> **Material do aluno** · Introdução à Programação · Linguagem C · **20 exercícios** · Continuação da lista de estruturas condicionais e funções

Objetivos de aprendizagem

Ao concluir esta lista, você deverá ser capaz de:

1. Construir repetições com `for`, `while` e `do while` e escolher a mais adequada para cada situação.
2. Diferenciar **contador**, **acumulador**, **condição de parada** e **valor sentinela**.
3. Combinar laços com `if`, `else if`, `else` e `switch-case`.
4. Desenvolver funções que usam repetição e retornam resultados ou executam ações.
5. Testar situações comuns, casos-limite e encerramentos imediatos.

Orientações para a turma

- Faça **um arquivo `.c` para cada exercício**, com **um único `main(void)` em cada arquivo**. Sugestões de nome: `exercicio01.c`, `exercicio02.c` etc.
- **Partes 1, 2 e 3:** use como laço principal a estrutura indicada na seção. **Parte 4:** combine as estruturas solicitadas.
- Nos exercícios que pedem função, **defina-a antes do `main()`** e chame-a dentro do `main()`.
- **Entradas são numéricas e bem-formadas**, salvo quando um exercício exigir expressamente validação de faixa ou categoria. Por exemplo, se o enunciado pede N de 1 a 30, você pode assumir que está nessa faixa, exceto se houver instrução para validar N. Não é necessário tratar o usuário que digita letras no lugar de números.
- Os números decimais dos exemplos usam **ponto** (`7.5`). Para evitar diferenças de configuração regional, utilize a configuração padrão de execução do seu ambiente.
- As **saídas essenciais** nos exemplos não incluem os textos das perguntas de entrada; você pode utilizar mensagens de solicitação próprias. Preserve os valores e regras definidos no enunciado.
- Use chaves `{ }` e identação em todos os laços e decisões. Não use vetores nem recursão, pois não são necessários nesta lista.
- **Para a prática:** evite copiar o gabarito antes de testar os casos sugeridos.

Como compilar no Windows (PowerShell)

Com o GCC instalado e configurado no `PATH`:

```
gcc --version
gcc -std=c11 -Wall -Wextra -Wpedantic exercicio01.c -o exercicio01.exe
.\exercicio01.exe
```

Cada exercício deve ser compilado **separadamente**. Caso `gcc` não seja reconhecido, o compilador deve ser instalado/configurado.

Mini-tutorial — Qual laço usar?

`for` — quando existe uma contagem conhecida: o início, a condição e a atualização aparecem juntos.

```
for (int i = 1; i <= 3; i++) {
    printf("%d\n", i);
}
```

`while` — quando precisamos conferir uma condição antes de cada repetição: o corpo pode executar **zero vezes**.

```
int saldo = 3;
while (saldo > 0) {
    printf("Restam %d unidades\n", saldo);
    saldo--;
}
```

`do while` — quando é preciso executar pelo menos uma vez: a condição só é verificada após o corpo.

```
int opcao;
do {
    printf("Digite 0 para sair: ");
    scanf("%d", &opcao);
} while (opcao != 0);
```

Três conceitos para não esquecer

- **Contador**: `quantidade++` registra quantas vezes algo aconteceu.
- **Acumulador**: `total += valor` soma os valores ao longo das iterações.
- **Parada**: um limite (`i <= n`), uma condição (`total < meta`) ou um **sentinela** (`numero != 0`) determina quando terminar.

> **Atenção:** todo laço deve ter um caminho para encerrar. Em `while` e `do while` , pense em **qual variável pode alterar a condição**. Nas entradas com `scanf(" %c", &categoria)` , o espaço antes de `%c` ignora quebras de linha pendentes.

Parte 1 — `for` (exercícios 01 a 05)

Nesta seção, utilize `for` como estrutura de repetição. Os problemas trabalham contagem, seleção, acumuladores e uma função no exercício 05.

Exercício 01 — Contagem crescente

Foco: `for` ; inicialização, condição e incremento.

Leia um inteiro **N de 1 a 30** e exiba os números de **1 até N**, inclusive. Cada número deve aparecer na mesma linha.

Entrada: Um inteiro N.

Exemplo:

```
Entrada:
5

Saída essencial:
Sequencia: 1 2 3 4 5
```

Testes que você deve fazer: N = 1: exibir somente 1. · N = 5: incluir o 5. · N = 30: terminar em 30.

Exercício 02 — Números pares até N

Foco: `for` combinado com `if` e operador `%` .

Leia um inteiro **N de 0 a 50** e exiba todos os **números pares de 0 até N**, inclusive. Considere que **zero é par**.

Entrada: Um inteiro N.

Exemplo:

```
Entrada:
9

Saída essencial:
Pares: 0 2 4 6 8
```

Testes que você deve fazer: N = 0: exibir 0. · N = 1: exibir somente 0. · N = 10: incluir 10.

Exercício 03 — Somatório de 1 até N

Foco: `for` ; variável acumuladora.

Leia um inteiro **N de 1 a 1000** e calcule a **soma de todos os inteiros de 1 até N** usando um laço. Não utilize fórmula matemática pronta.

Entrada: Um inteiro N.

Exemplo:

```
Entrada:
5

Saída essencial:
Soma: 15
```

Testes que você deve fazer: N = 1: soma 1. · N = 5: soma 15. · N = 1000: soma 500500.

Exercício 04 — Tabuada de um número

Foco: `for` ; multiplicação; saída organizada.

Leia um número inteiro **de -10 a 10** e exiba sua tabuada, de **1 até 10**, no formato `numero x multiplicador = resultado` .

Entrada: Um inteiro.

Exemplo:

```
Entrada:
3

Saída essencial:
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30
```

Testes que você deve fazer: Número 0: todos os resultados serão 0. · Número -2: resultados negativos. · Número 10: terminar em 100.

Exercício 05 — Desafio com função — Fatorial

Foco: Função com retorno `int` ; `for` ; caso especial `0!` .

Crie a função **`int calcularFatorial(int n)`** que calcule `n!` por multiplicações sucessivas usando `for` . No `main()` , leia **N de 0 a 12**, chame a função e mostre o resultado. Lembre-se de que **`0! = 1`** . O limite 12 evita estouro de `int` de 32 bits.

Entrada: Um inteiro N de 0 a 12.

Exemplo:

```
Entrada:
5

Saída essencial:
Fatorial de 5: 120
```

Testes que você deve fazer: N = 0: 1. · N = 1: 1. · N = 12: 479001600.

Parte 2 — `while` (exercícios 06 a 10)

Nesta seção, utilize `while` como laço principal. Preste atenção à condição de parada, ao valor sentinela e à atualização das variáveis.

Exercício 06 — Contagem regressiva

Foco: `while` ; condição verificada antes de cada volta.

Leia **N de 0 a 30** e exiba uma contagem regressiva, começando em N e terminando em zero. Faça a repetição com `while` .

Entrada: Um inteiro N.

Exemplo:

```
Entrada:
4

Saída essencial:
Contagem: 4 3 2 1 0
```

Testes que você deve fazer: N = 0: exibir só 0. · N = 1: 1 e 0. · N = 7: incluir zero.

Exercício 07 — Soma até digitar zero

Foco: `while` ; valor sentinela, contador e acumulador.

Leia uma sequência de inteiros (cada um entre **-1000 e 1000**). Some os valores digitados e conte quantos números foram informados. **Quando o usuário digitar zero, encerre a leitura**: o zero não entra na contagem nem na soma. A primeira entrada já pode ser zero.

Entrada: Uma sequência de inteiros terminada em 0.

Exemplo:

```
Entrada:
7
-2
5
0

Saída essencial:
Quantidade: 3
Soma: 10
```

Testes que você deve fazer: Entrada 0: quantidade 0 e soma 0. · Entrada 3, -3, 0: soma 0, quantidade 2. · Entrada -5, 0: soma -5.

Exercício 08 — Senha com três tentativas

Foco: `while` ; condição composta; `if-else` .

Simule a verificação de uma senha **numérica fictícia** de valor `2468` . O usuário pode tentar **no máximo três vezes**. Se acertar, informe em qual tentativa ocorreu o acerto e encerre. Se errar as três, mostre **"Acesso bloqueado."**. Este exercício **não representa uma autenticação segura**.

Entrada: Até três inteiros, um por tentativa.

Exemplo:

```
Entrada:
1111
2468

Saída essencial:
Senha incorreta.
Acesso permitido na tentativa 2.
```

Testes que você deve fazer: Acertar na primeira: apenas uma tentativa. · Acertar na terceira: liberar. · Três erros: bloquear.

Exercício 09 — Meta de leitura

Foco: `while` ; repetição até alcançar uma meta.

Leia uma **meta positiva** de páginas (de 1 a 1000). Em seguida, leia quantas páginas foram lidas em cada dia (**de 1 a 100**) até que o total seja **maior ou igual à meta**. Exiba quantos dias foram necessários e a quantidade total lida. É permitido ultrapassar a meta no último dia.

Entrada: Meta e, em seguida, quantidades diárias até atingir a meta.

Exemplo:

```
Entrada:
10
4
3
6

Saída essencial:
Dias: 3
Paginas lidas: 13
```

Testes que você deve fazer: Meta 5 e dia 5: 1 dia. · Meta 10 e dias 4, 3, 6: 3 dias, 13 páginas. · Meta 1 e dia 100: 1 dia, 100 páginas.

Exercício 10 — Desafio com função — Quantidade de dígitos

Foco: Função `int contarDigitos(int numero)` ; divisão inteira; `while` .

Implemente **`int contarDigitos(int numero)`** para retornar quantos algarismos decimais existem em um inteiro **de 0 a 1.000.000.000**. Use sucessivas divisões inteiras por 10 dentro de `while` ; **zero possui um dígito**. Chame a função em `main()` .

Entrada: Um inteiro não negativo até 1.000.000.000.

Exemplo:

```
Entrada:
5020

Saída essencial:
Digitos: 4
```

Testes que você deve fazer: 0: 1 dígito. · 9: 1 dígito. · 1000: 4 dígitos. · 1000000000: 10 dígitos.

Parte 3 — `do while` (exercícios 11 a 15)

Nesta seção, o laço principal deve ser `do while`. Explore menus, entrada validada e situações que exigem ao menos uma execução.

Exercício 11 — Validando uma nota

Foco: `do while`; validação de intervalo; `if` opcional.

Solicite uma nota real até o usuário informar um valor **entre 0 e 10, inclusive**. Se estiver fora da faixa, mostre **"Nota invalida."** e peça novamente. Ao final, exiba a nota aceita com uma casa decimal. Use obrigatoriamente `do while`.

Entrada: Uma ou mais notas reais; a última deve estar entre 0 e 10.

Exemplo:

```
Entrada:
-1
11
7.5

Saída essencial:
Nota invalida.
Nota invalida.
Nota aceita: 7.5
```

Testes que você deve fazer: 0: aceita. · 10: aceita. · -1, 11, 7.5: aceita só a última.

Exercício 12 — Menu de lanchonete

Foco: `do while`; `switch-case`; opção de saída.

Crie um menu que seja apresentado **pelo menos uma vez** e reapareça após cada escolha: `1` — Sanduíche, `2` — Suco, `0` — Encerrar. Mostre o item correspondente a cada pedido; para qualquer outro código, mostre **"Opcao invalida."**. Ao escolher 0, exiba **"Atendimento encerrado."** e finalize.

Entrada: Uma sequência de opções inteiras, terminada em 0.

Exemplo:

```
Entrada:
1
9
2
0

Saída essencial:
Pedido: Sanduiche
Opcao invalida.
Pedido: Suco
Atendimento encerrado.
```

Testes que você deve fazer: 0 como primeira opção: encerrar. · 1, 2, 0: mostrar os dois pedidos. · 9, 0: informar opção inválida.

Exercício 13 — Jogo de adivinhação

Foco: `do while` e `if / else if / else`.

O computador possui um número secreto **fixo: 37** (para permitir testes reproduzíveis). Peça palpites inteiros **de 1 a 100** até o jogador acertar. Após cada erro, informe **"Muito baixo."** ou **"Muito alto."**. Quando acertar, informe **o número de tentativas**. Utilize `do while`.

Entrada: Um ou mais inteiros entre 1 e 100, terminando no acerto.

Exemplo:

```
Entrada:
20
50
37

Saída essencial:
Muito baixo.
Muito alto.
Acertou em 3 tentativa(s).
```

Testes que você deve fazer: 37: acerto em 1 tentativa. · 36, 37: primeiro palpite muito baixo. · 38, 37: primeiro palpite muito alto.

Exercício 14 — Pesquisa de satisfação

Foco: `do while` ; sentinela, validação e média.

Leia avaliações **inteiras de 1 a 5** até a digitação de **0 (encerrar)**. Valores diferentes de 0, 1, 2, 3, 4 e 5 são inválidos: mostre **"Avaliacao invalida."** e desconsidere-os. Ao final, mostre o total de avaliações válidas e a média com duas casas decimais. Se não houver avaliações, mostre **"Sem avaliacoes."**, sem dividir por zero.

Entrada: Uma sequência de números inteiros; 0 encerra.

Exemplo:

```
Entrada:
5
9
3
0

Saída essencial:
Avaliacao invalida.
Avaliacoes: 2
Media: 4.00
```

Testes que você deve fazer: 0: sem avaliações. · 5, 3, 0: média 4.00. · -1, 0: entrada inválida, depois sem avaliações.

Exercício 15 — Desafio com função — Escolha validada

Foco: Função `int lerOpcao(void)` ; `do while` ; `switch` .

Crie **`int lerOpcao(void)`** . A função deverá solicitar um nível (`1` — Básico, `2` — Intermediário, `3` — Avançado), **repetir a leitura** até receber 1, 2 ou 3, e retornar a opção válida. O `main()` deverá chamar a função e usar `switch` para imprimir o nome do nível selecionado.

Entrada: Uma ou mais opções inteiras; a última deve estar em 1, 2 ou 3.

Exemplo:

```
Entrada:
0
4
2

Saída essencial:
Opcao invalida.
Opcao invalida.
Nivel: Intermediario
```

Testes que você deve fazer: 1: Básico. · 2: Intermediário. · 3: Avançado. · 0, 4, 2: ignorar inválidas e aceitar 2.

Parte 4 — Desafios integradores (exercícios 16 a 20)

Agora combine `for`, `while`, `do while`, `if/else`, `switch-case` e funções conforme cada enunciado. O objetivo é identificar qual estrutura resolve cada parte do problema.

Exercício 16 — Relatório de notas da turma

Foco: `for` + `do while` + `if-else`; acumulador.

Leia a quantidade **N de alunos (1 a 30)**. Para cada aluno, leia uma nota real **de 0 a 10**; caso esteja fora da faixa, peça a nota daquele aluno novamente. Conte quantos alunos foram **aprovados (nota >= 7.0)** e **reprovados (nota < 7.0)**. Exiba também a **média da turma com duas casas**. Não utilize vetores.

Entrada: N e, em seguida, N notas válidas; pode haver notas inválidas antes das válidas.

Exemplo:

```
Entrada:
3
7
5
10

Saída essencial:
Aprovados: 2
Reprovados: 1
Media da turma: 7.33
```

Testes que você deve fazer: Uma nota igual a 7: aprovado. · Nota 6.9: reprovado. · Nota -1, depois 8: deve pedir a mesma nota novamente.

Exercício 17 — Caixa eletrônico simplificado

Foco: `while` + `switch-case` + `if-else` .

Simule uma conta com **saldo inicial de R$ 100,00**. Repita um menu até o usuário selecionar **0 — Sair**: **1 — Depositar**, **2 — Sacar**, **3 — Consultar saldo**. Depósitos e saques devem possuir valores **maiores que zero**; saques não podem exceder o saldo. Para entradas inválidas, exiba uma mensagem e **não altere o saldo**. Ao encerrar, mostre o saldo final com duas casas decimais. O uso de `float` é apenas didático; sistemas financeiros reais precisam de representação monetária apropriada.

Entrada: Opções inteiras e, quando necessário, valores reais; termine em 0.

Exemplo:

```
Entrada:
1
50
2
30
3
0

Saída essencial:
Deposito realizado.
Saque realizado.
Saldo atual: R$ 120.00
Saldo final: R$ 120.00
```

Testes que você deve fazer: 0 imediatamente: saldo final 100.00. · Saque de 101: saldo insuficiente. · Depósito de 0: valor inválido. · Opção 9: opção inválida.

Exercício 18 — Verificador de número primo

Foco: Função `int ehPrimo(int numero)` ; `for` ; `if` .

Crie a função **`int ehPrimo(int numero)`** para retornar **1 se o número for primo** e **0 caso contrário**. Um primo é um inteiro maior que 1 com exatamente dois divisores positivos (1 e ele mesmo). Leia um inteiro **N de 0 a 1000** e use a função para mostrar **"Primo."** ou **"Nao primo."**. Utilize `for` e `%` na verificação.

Entrada: Um inteiro N de 0 a 1000.

Exemplo:

```
Entrada:
17

Saída essencial:
Primo.
```

Testes que você deve fazer: 0 e 1: não primos. · 2: primo. · 9: não primo. · 97: primo.

Exercício 19 — Menu de relatórios com funções

Foco: `do while` + `switch` + funções com `for` .

Crie um menu que repita até **0 — Sair**: **1 — Somar de 1 até N** e **2 — Exibir os pares de 0 até N**. Para cada opção 1 ou 2, leia **N entre 1 e 20** (assuma esse intervalo válido), calcule ou exiba o resultado usando funções: `int somarAte(int n)` e `void mostrarPares(int n)` . Implemente os dois cálculos com `for` . Para opção desconhecida, mostre **"Opcao invalida."**.

Entrada: Uma sequência de opções; após opções 1 e 2, um N de 1 a 20; termine em 0.

Exemplo:

```
Entrada:
1
4
2
6
0

Saída essencial:
Soma: 10
Pares: 0 2 4 6
Encerrado.
```

Testes que você deve fazer: 0 imediatamente: encerrar. · 1, 1, 0: soma 1. · 2, 1, 0: exibir apenas 0. · 9, 0: opção inválida.

Exercício 20 — Desafio final — Controle de vendas

Foco: `do while` , `for` , `switch` , `if` e função com retorno.

Uma loja registra **N vendas (1 a 10)**. Valide N com `do while` até que esteja no intervalo. Para cada venda, leia o **preço unitário positivo**, a **quantidade inteira de 1 a 100**, e uma categoria de cliente: **A = 10% de desconto**, **B = 5% de desconto**, **C = sem desconto**. A categoria deve ser **A, B ou C maiúsculo**; caso contrário, repita sua leitura com `do while` . Implemente `float calcularTotal(float preco, int quantidade, char categoria)` , usando `switch` , para calcular o valor final daquela venda. Ao terminar, mostre o valor de cada venda, o **faturamento total** e **quantas vendas receberam desconto**. Para fins didáticos, use `float` e mostre valores com duas casas; não se trata de um sistema financeiro de produção.

Entrada: N, seguido de N trios preço/quantidade/categoria, com repetições para categorias inválidas.

Exemplo:

```
Entrada:
2
100
2
A
50
1
C

Saída essencial:
Venda 1: R$ 180.00
Venda 2: R$ 50.00
Faturamento: R$ 230.00
Vendas com desconto: 1
```

Testes que você deve fazer: N inválido 0, depois 1: aceitar 1. · Categoria X, depois B: exigir nova categoria. · A: 10%; B: 5%; C: 0%. · N=1, preço 10, quantidade 1, categoria C: total R$ 10.00.

Checklist

- ☐ Criei **um arquivo por exercício**, cada um com seu `main(void)`.
- ☐ Usei os laços pedidos e não deixei repetições sem condição de parada.
- ☐ Conferi **limites** (0, 1, último valor incluído) e **encerramentos imediatos**.
- ☐ Para menus, testei a saída pela opção 0 e opções inválidas.
- ☐ Para funções, conferi os parâmetros, o tipo do retorno e a chamada em `main()`.
- ☐ Compilei cada arquivo e li eventuais avisos do compilador.

Para pensar antes de ver o gabarito

1. Quando o `while` executa zero vezes, mas o `do while` executa uma?
2. Qual é a diferença entre contar valores lidos e somar esses valores?
3. Por que, em um menu, é importante atualizar a opção antes de testar a próxima volta?
4. Quando faz sentido combinar `for` com `if` ou `switch`?
5. Em qual exercício a falta de uma condição de parada geraria um laço infinito?